Artificial Intelligence Techniques *in* Manufacturing

Convolution

Infinite array

$$y = x * w \rightarrow y[i] = \sum_{k=-\infty}^{\infty} x[i-k]w[k]$$

Infinite array

$$y = x * w \rightarrow y[i] = \sum_{k=-\infty}^{\infty} x[i-k]w[k]$$

$i = 0$

$$y[0] = \sum_{k=-\infty}^{\infty} x[-k]w[k]$$

$$\ldots + x[-1]w[1] + x[0]w[0] + x[1]w[-1] + \cdots$$

Infinite array

$$y = x * w \rightarrow y[i] = \sum_{k=-\infty}^{\infty} x[i-k]w[k]$$

$i = 0$

$$y[0] = \sum_{k=-\infty}^{\infty} x[-k]w[k]$$

$i = 1$

$$y[1] = \sum_{k=-\infty}^{\infty} x[1-k]w[k]$$

$$\ldots + x[-1]w[1] + x[0]w[0] + x[1]w[-1] + \cdots$$

$$\ldots + x[0]w[1] + x[1]w[0] + x[2]w[-1] + \cdots$$

Infinite array

$$y = x * w \rightarrow y[i] = \sum_{k=-\infty}^{\infty} x[i-k]w[k]$$

$i = -1$

$$y[-1] = \sum_{k=-\infty}^{\infty} x[-1-k]w[k]$$

$$\ldots + x[-2]w[1] + x[-1]w[0] + x[0]w[-1] + x[1]w[-2]$$

$i = 0$

$$y[0] = \sum_{k=-\infty}^{\infty} x[-k]w[k]$$

$$\ldots + x[-1]w[1] + x[0]w[0] + x[1]w[-1] + \cdots$$

$i = 1$

$$y[1] = \sum_{k=-\infty}^{\infty} x[1-k]w[k]$$

$$\ldots + x[0]w[1] + x[1]w[0] + x[2]w[-1] + \cdots$$

Finite array

$$y = x * w \rightarrow y[i] = \sum_{k=0}^{m-1} x[i+m-k-1]w[k]$$

$m \rightarrow$ Size of w array

Finite array

$$y = x * w \rightarrow y[i] = \sum_{k=0}^{m-1} x[i+m-k-1]w[k]$$

$m \rightarrow$ Size of w array

Example

Vector x

x_0	x_1	x_2	x_3	x_4	x_5

$n = 6$

Kernel w

w_0	w_1	w_2	w_3

$m = 4$

$$y[i] = \sum_{k=0}^{3} x[i+3-k]w[k]$$

$i = 0$

$$y[0] = \sum_{k=0}^{3} x[3-k]w[k]$$

x_0	x_1	x_2	x_3	x_4	x_5

w_0	w_1	w_2	w_3

$$y[0] = x[3]w[0] + x[2]w[1] + x[1]w[2] + x[0]w[3]$$

$i = 0$

$$y[0] = \sum_{k=0}^{3} x[3-k]w[k]$$

x_0	x_1	x_2	x_3	x_4	x_5

w_0	w_1	w_2	w_3

$$y[0] = x[3]w[0] + x[2]w[1] + x[1]w[2] + x[0]w[3]$$

$i = 1$

$$y[1] = \sum_{k=0}^{3} x[4-k]w[k]$$

x_0	x_1	x_2	x_3	x_4	x_5

w_0	w_1	w_2	w_3

$$y[0] = x[4]w[0] + x[3]w[1] + x[2]w[2] + x[1]w[3]$$

$i = 0$

$$y[0] = \sum_{k=0}^{3} x[3-k]w[k]$$

x_0	x_1	x_2	x_3	x_4	x_5
w_0	w_1	w_2	w_3		

$$y[0] = x[3]w[0] + x[2]w[1] + x[1]w[2] + x[0]w[3]$$

$i = 1$

$$y[1] = \sum_{k=0}^{3} x[4-k]w[k]$$

x_0	x_1	x_2	x_3	x_4	x_5
	w_0	w_1	w_2	w_3	

$$y[0] = x[4]w[0] + x[3]w[1] + x[2]w[2] + x[1]w[3]$$

$i = 2$

$$y[2] = \sum_{k=0}^{3} x[5-k]w[k]$$

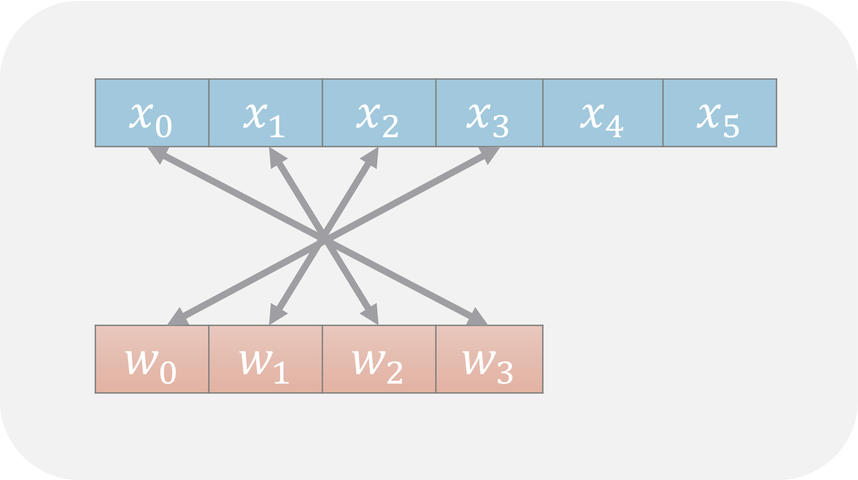
x_0	x_1	x_2	x_3	x_4	x_5
		w_0	w_1	w_2	w_3

$$y[0] = x[5]w[0] + x[4]w[1] + x[3]w[2] + x[2]w[3]$$

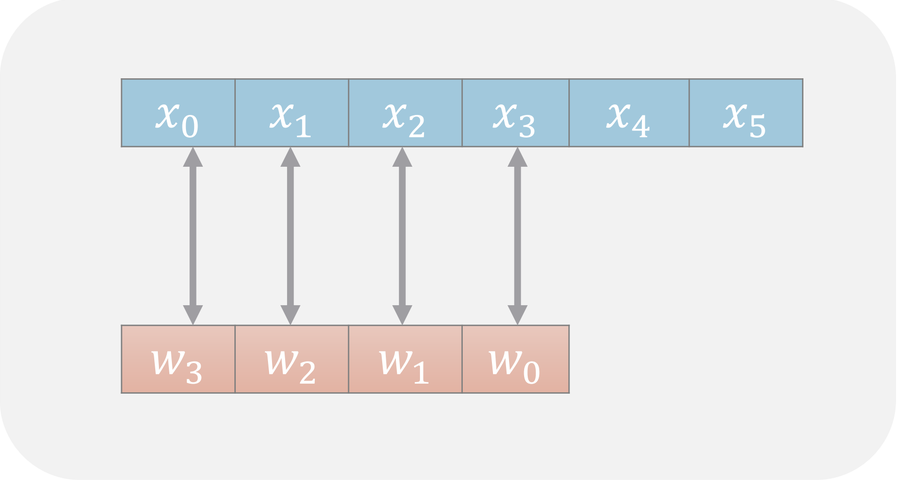
Reversing filter

$$y[0] = x[3]w[0] + x[2]w[1] + x[1]w[2] + x[0]x[3]$$

Filter w

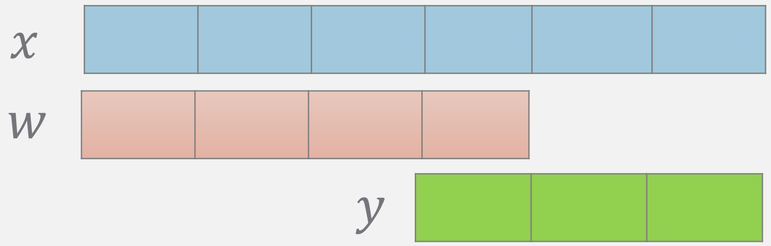


Filter w^r



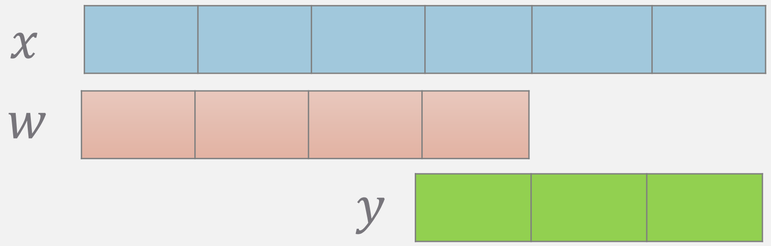
Padding

$p = 0$

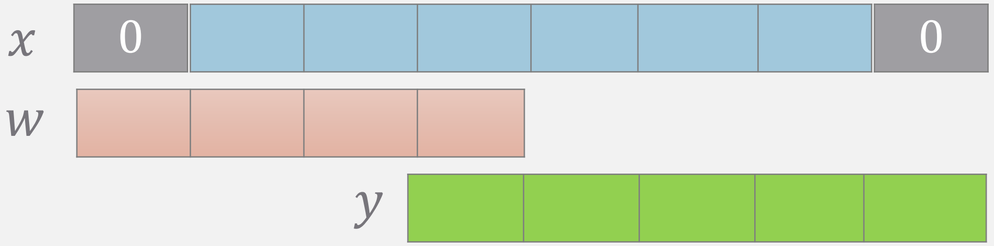


Padding

$p = 0$

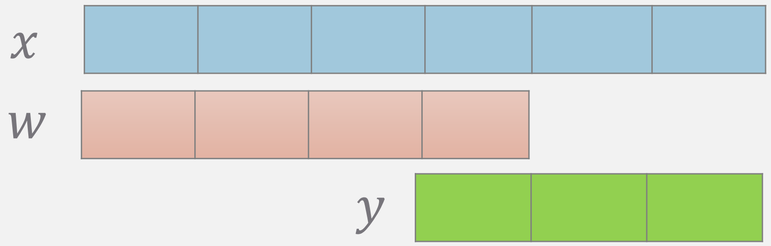


$p = 1$

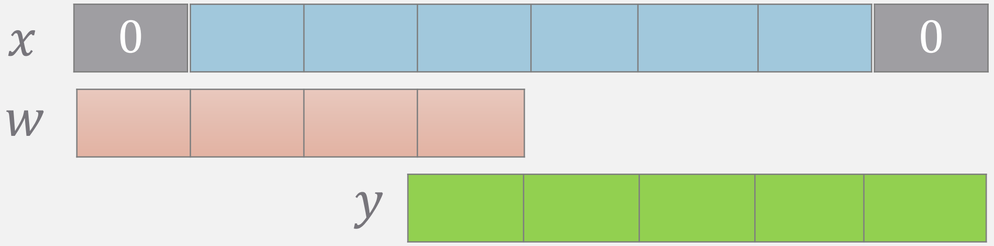


Padding

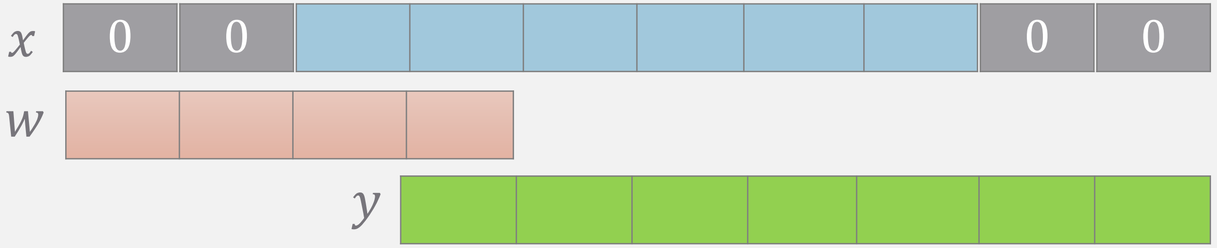
$p = 0$



$p = 1$

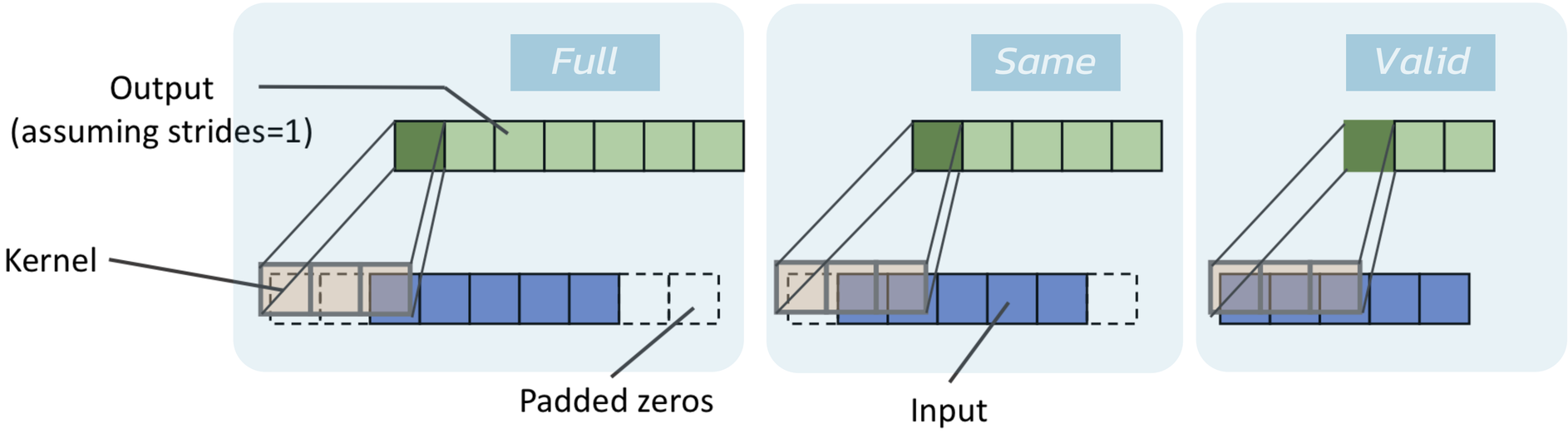


$p = 2$

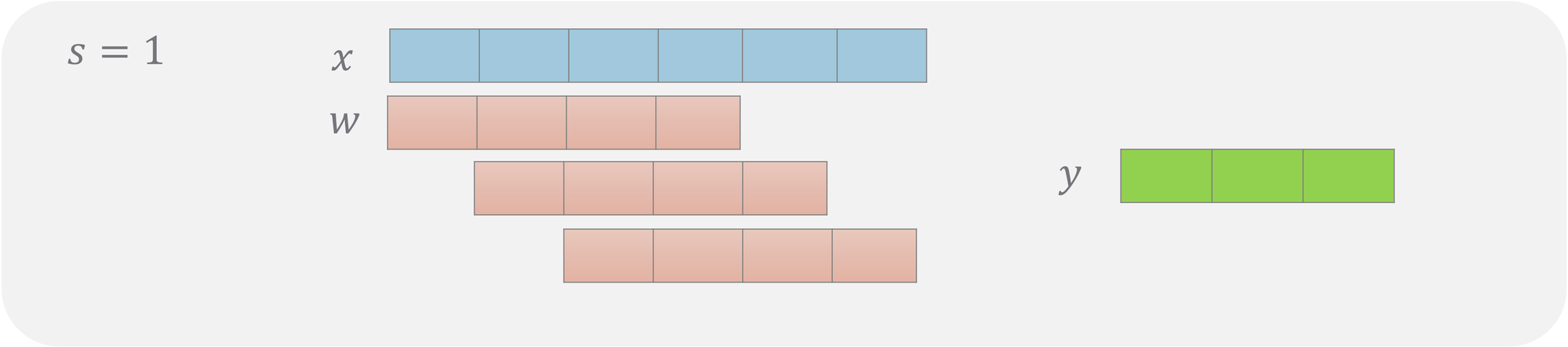


Padding types

Type	p	Note
Full	$p = m - 1$	• Increased dimension • Rarely used in CNN
Same	$p = f(m, n)$	• Equal dimension • Commonly used in CNN
Valid	$p = 0$	• Reduce dimension

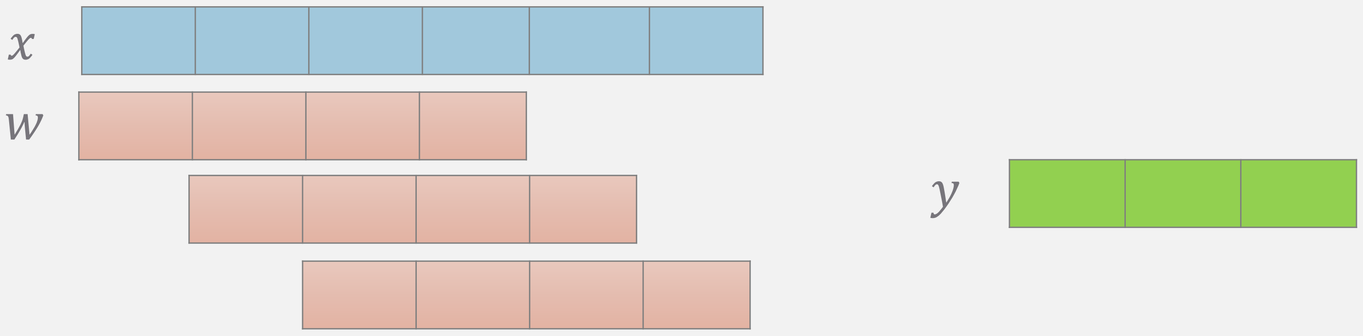


Stride

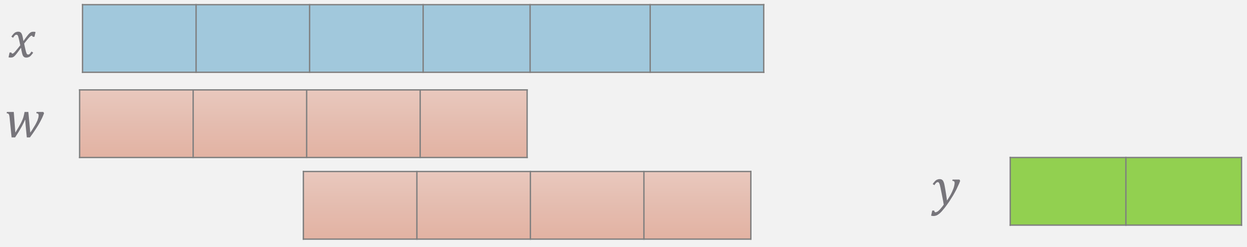


Stride

$s = 1$



$s = 2$



Output size

$$O_n = \text{floor}\left[\frac{n + 2p - m}{s}\right] + 1$$

- $n \rightarrow$ Input size
- $p \rightarrow$ Padding
- $m \rightarrow$ Filter size
- $s \rightarrow$ Stride

n	p	m	s	O_n
10	2	5	1	**10**
10	2	3	2	**6**

Same padding

Cross-correlation

- Convolution
  - $y = x * w \rightarrow y[i] = \sum_{k=-\infty}^{\infty} x[i - k]w[k]$
- Cross-correlation
  - $y = x * w \rightarrow y[i] = \sum_{k=-\infty}^{\infty} x[i + k]w[k]$
  - No need to "invert" the filter
- Most deep learning uses *cross-correlation*.
  - But refer to this operation as *convolution*.

2D Convolution

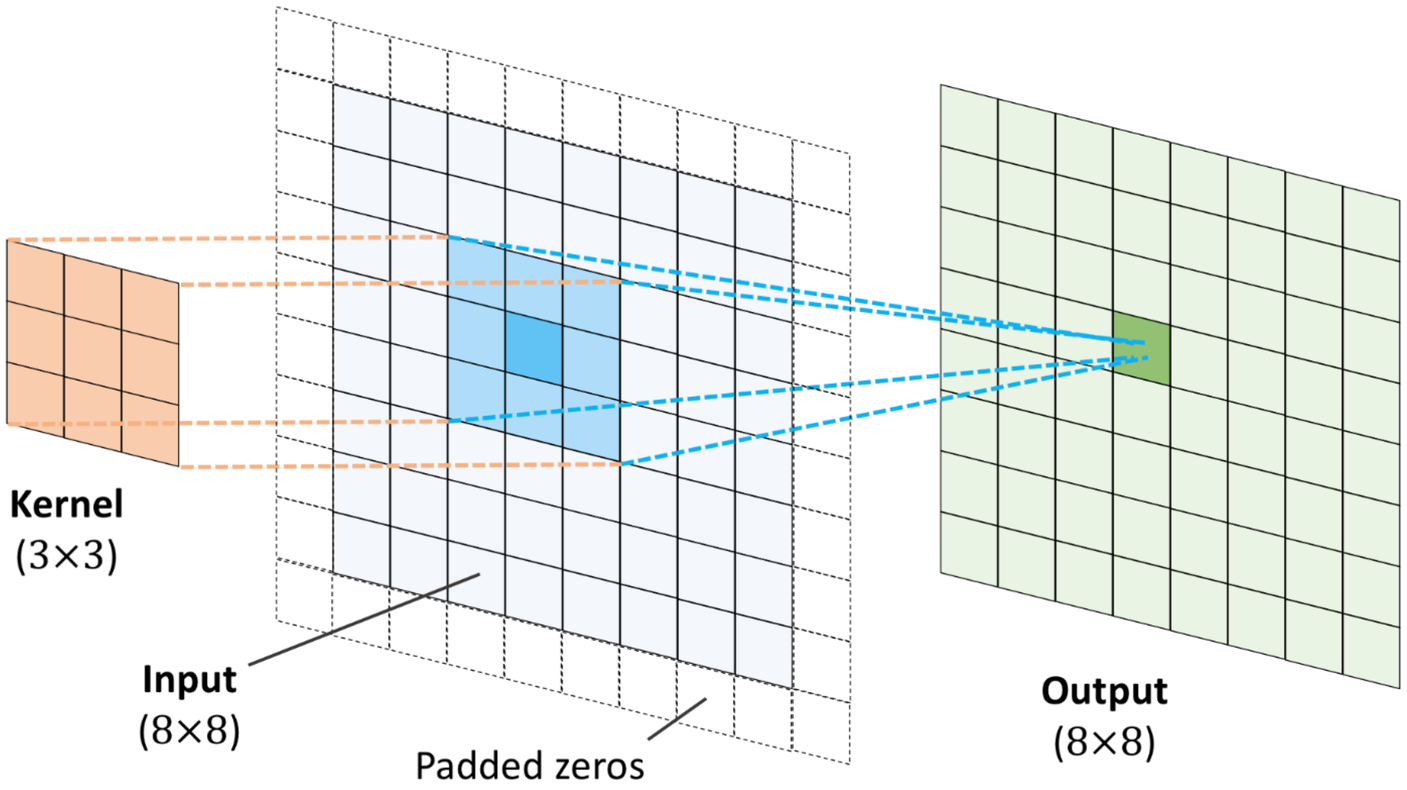
1D

$$y = x * w \rightarrow y[i] = \sum_{k=-\infty}^{\infty} x[i-k]w[k]$$

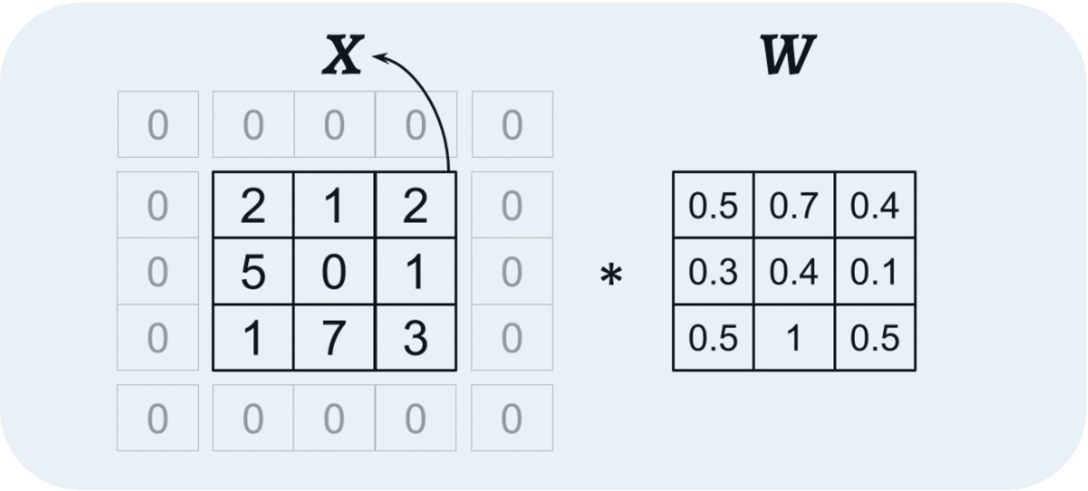
2D

$$y = x * w \rightarrow y[i,j] = \sum_{k_1=-\infty}^{\infty} \sum_{k_2=-\infty}^{\infty} x[i-k_1, j-k_2]w[k_1, k_2]$$

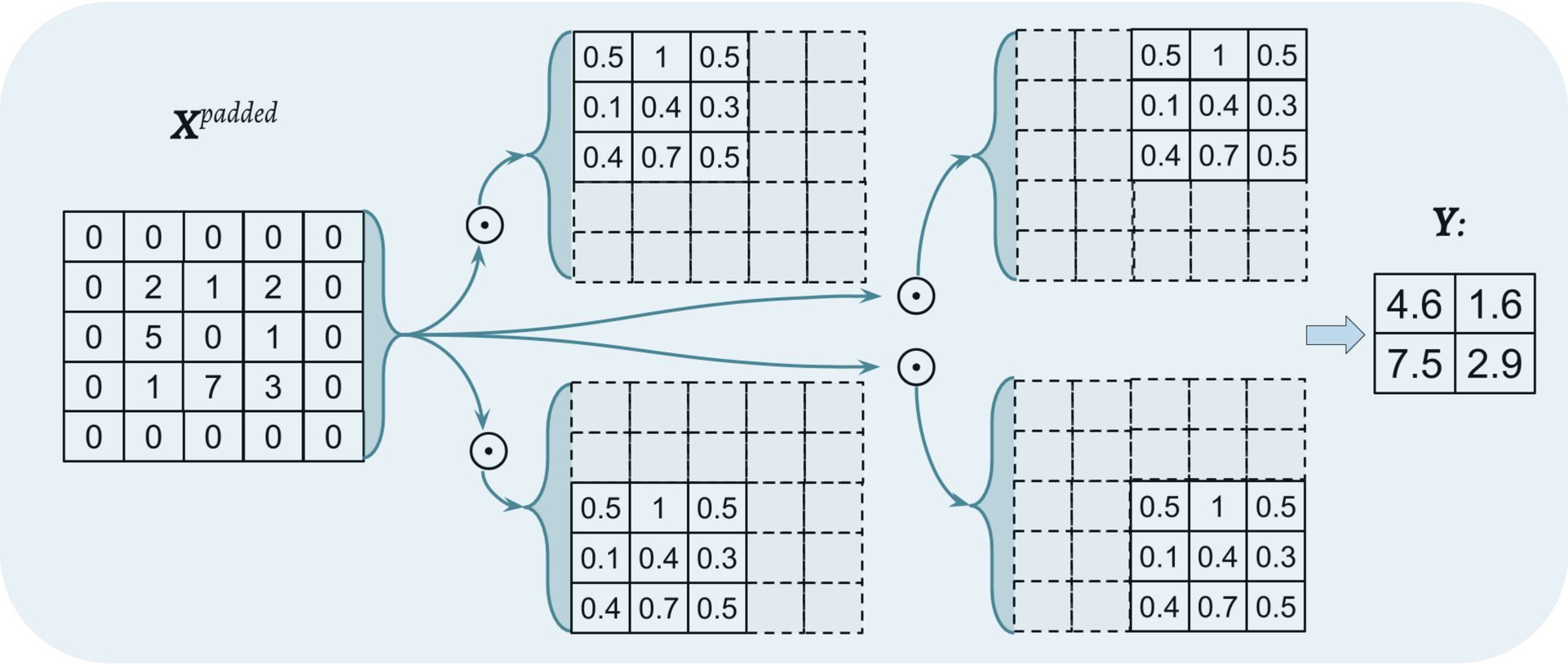
2D Convolution



- $n = 3 \times 3$
- $p = 1, 1$
- $m = 3 \times 3$
- $s = 2, 2$



$$\boldsymbol{W}^r = \begin{bmatrix} 0.5 & 1 & 0.5 \\ 0.1 & 0.4 & 0.3 \\ 0.4 & 0.7 & 0.5 \end{bmatrix}$$



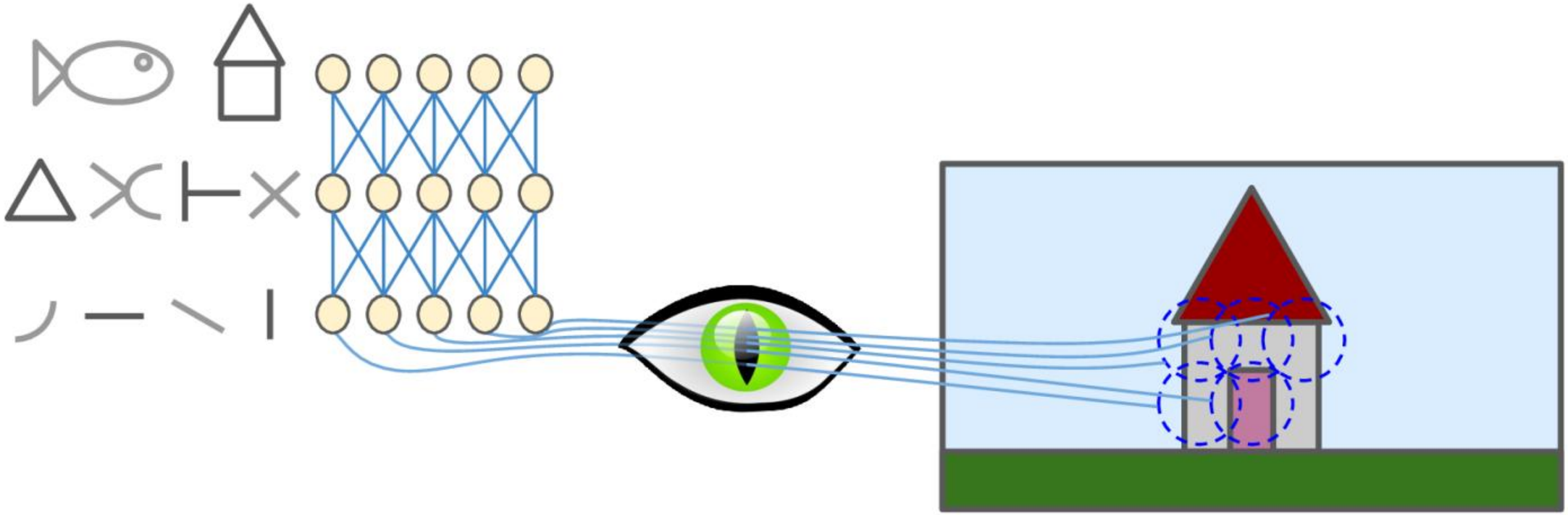
CNN

Visual recognition

- Human can recognize a puppy in the picture *effortlessly*.
- How can a computer recognize visual objects?

Convolutional neural network

- CNN emerged from study of brain's visual cortex. *(เปลือกสมองส่วนการเห็น)*
- Neurons in visual cortex
  - have *local receptive field*.
  - react only to visual stimuli located in a *limited region* of the visual field.
  - react to patterns that are combination of lower-level patterns.

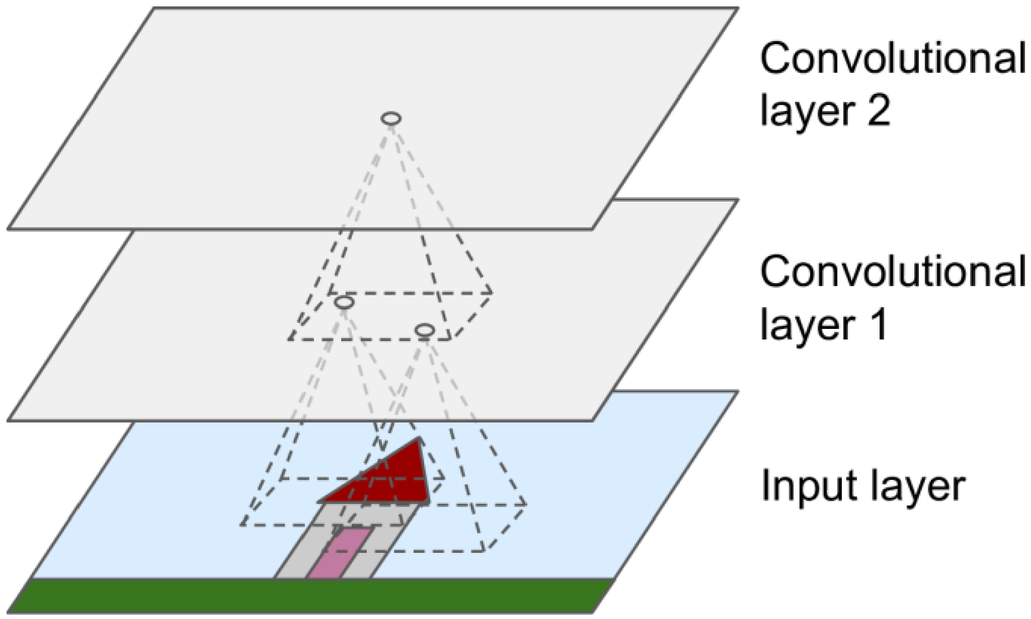


Convolutional neural network

- Visual cortex studies leads to *neocognition* paper in 1980.
  - Gradually leads to convolutional neural network.
- LeCun et al. introduce the *LeNet-5* architecture in 1998.
  - Modern implementation of CNN.
  - Employing back-propagation algorithm.

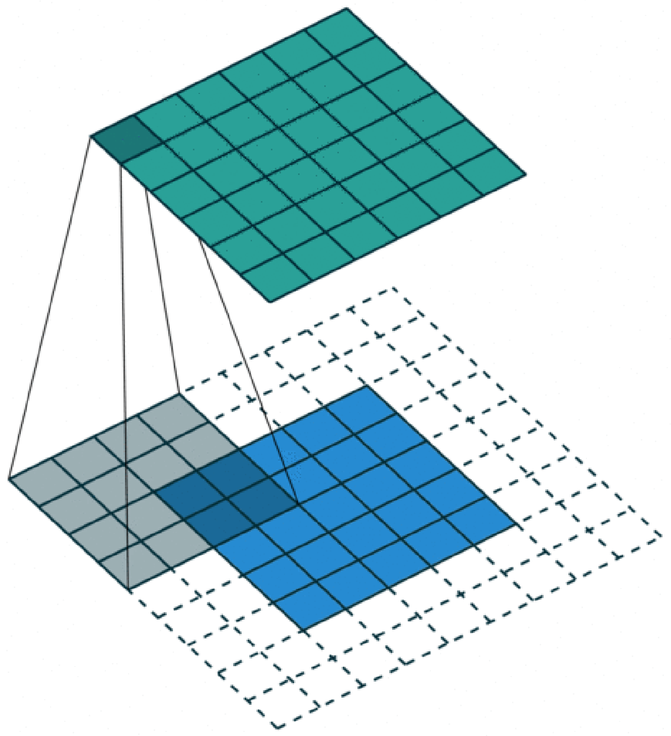
Yann LeCun in 2018

Convolutional layer

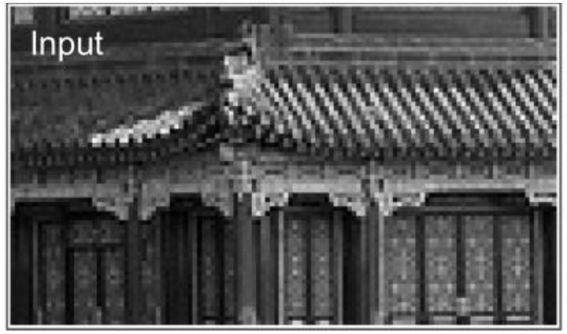


Kernels

- 2D array of weights
  - Learned parameters
- “Scan” through the layer.
  - Convolution operation.
- Produce an output
  - Called a “feature map”.
- Analogous to neuron’s receptive field

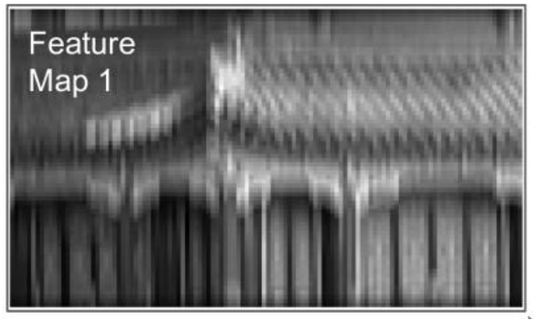


https://github.com/vdumoulin/conv_arithmetic



Vertical kernel

			1			
			1			
			1			
			1			
			1			
			1			
			1			



Horizontal kernel

1	1	1	1	1	1	1

Blank cells contain zeros.

Advantage of CNN Layers

Compared with fully connected layer

- Reduced parameters
  - Kernel are “shared” with multiple neurons in the layers.
  - Parameters does not increase with increased image size.
- Consider a color image with 150x100 pixels
  - A convolutional layer with 200 feature maps need ~15,000 parameters.
  - Fully connected layers of the same size will need ~674 million parameters.

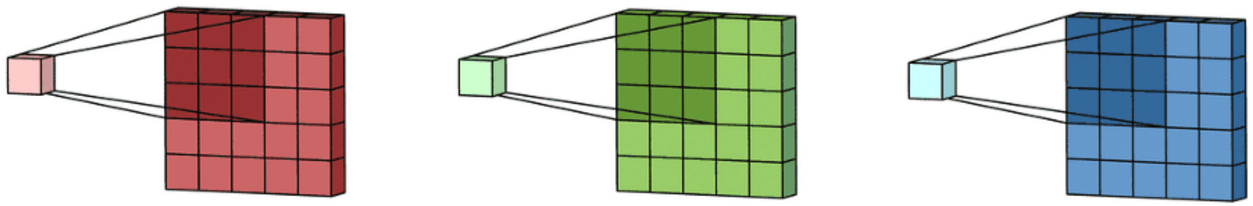
Advantage of CNN Layers

Compared with fully connected layer

- Translation invariant.
  - Due to scanning, location of features does not matter

Channels

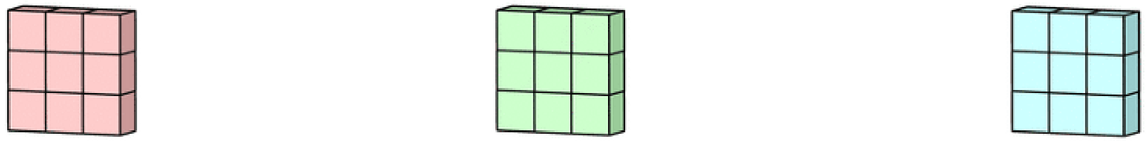
- A color image has 3 channels.
  - Neurons in the input layer become a 3D array.
  - *Width x Height x Channel*
- Need 3 kernels



https://towardsdatascience.com/a-comprehensive-introduction-to-different-types-of-convolutions-in-deep-learning-669281e58215

Channels

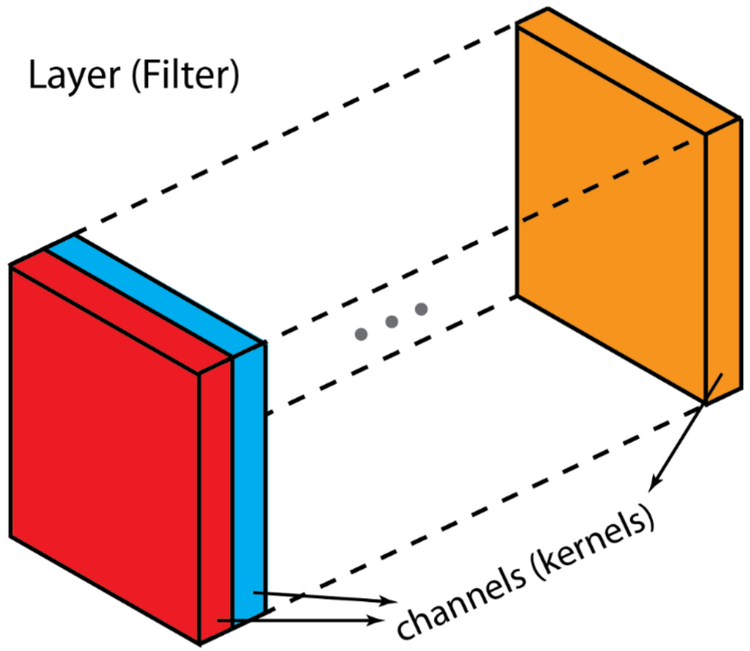
- The result is summed to obtain a single feature map (yellow).



https://towardsdatascience.com/a-comprehensive-introduction-to-different-types-of-convolutions-in-deep-learning-669281e58215

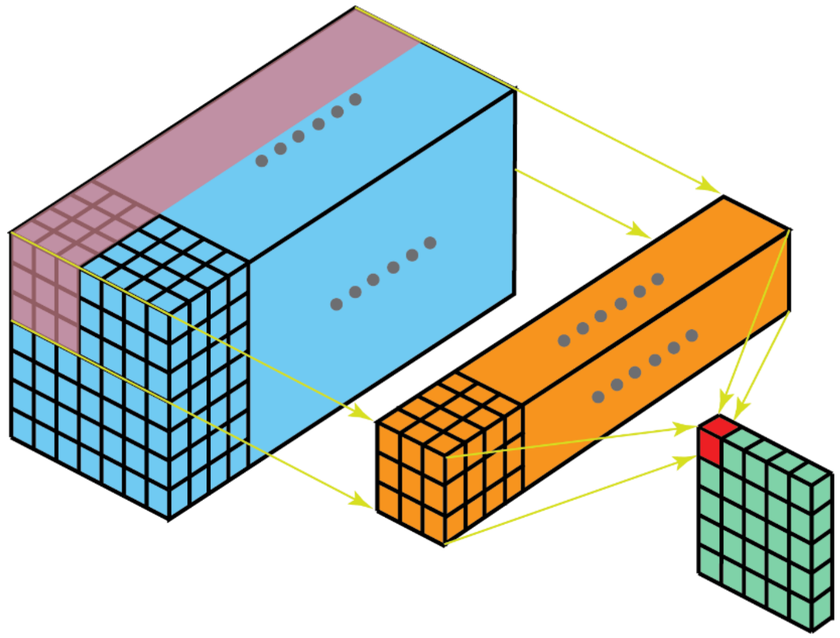
Filters

- Multiple kernels stacks together.
- Filters are 3D arrays.

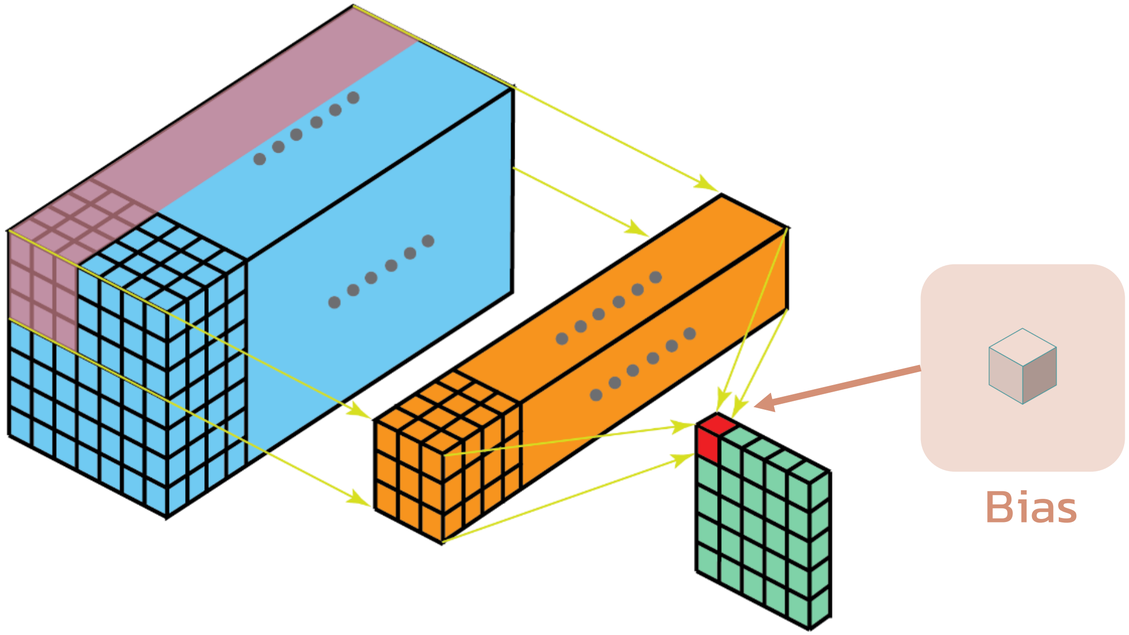


Filters

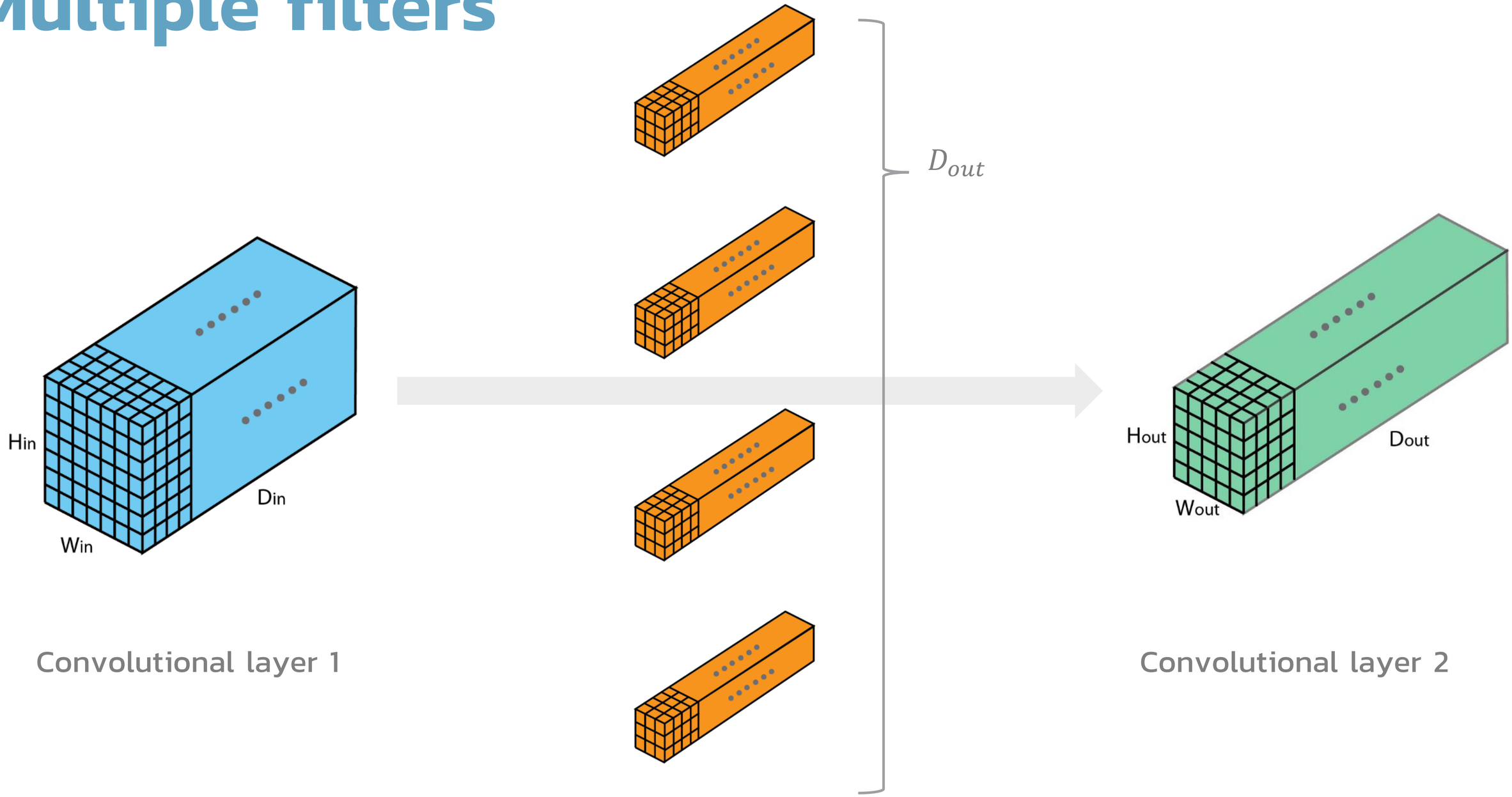
- The input layer and the filter have the same depth
  - Channel number = kernel number
- The 3D filter moves only in 2-direction
  - Height and width of the image
- An output is a *single* channel.



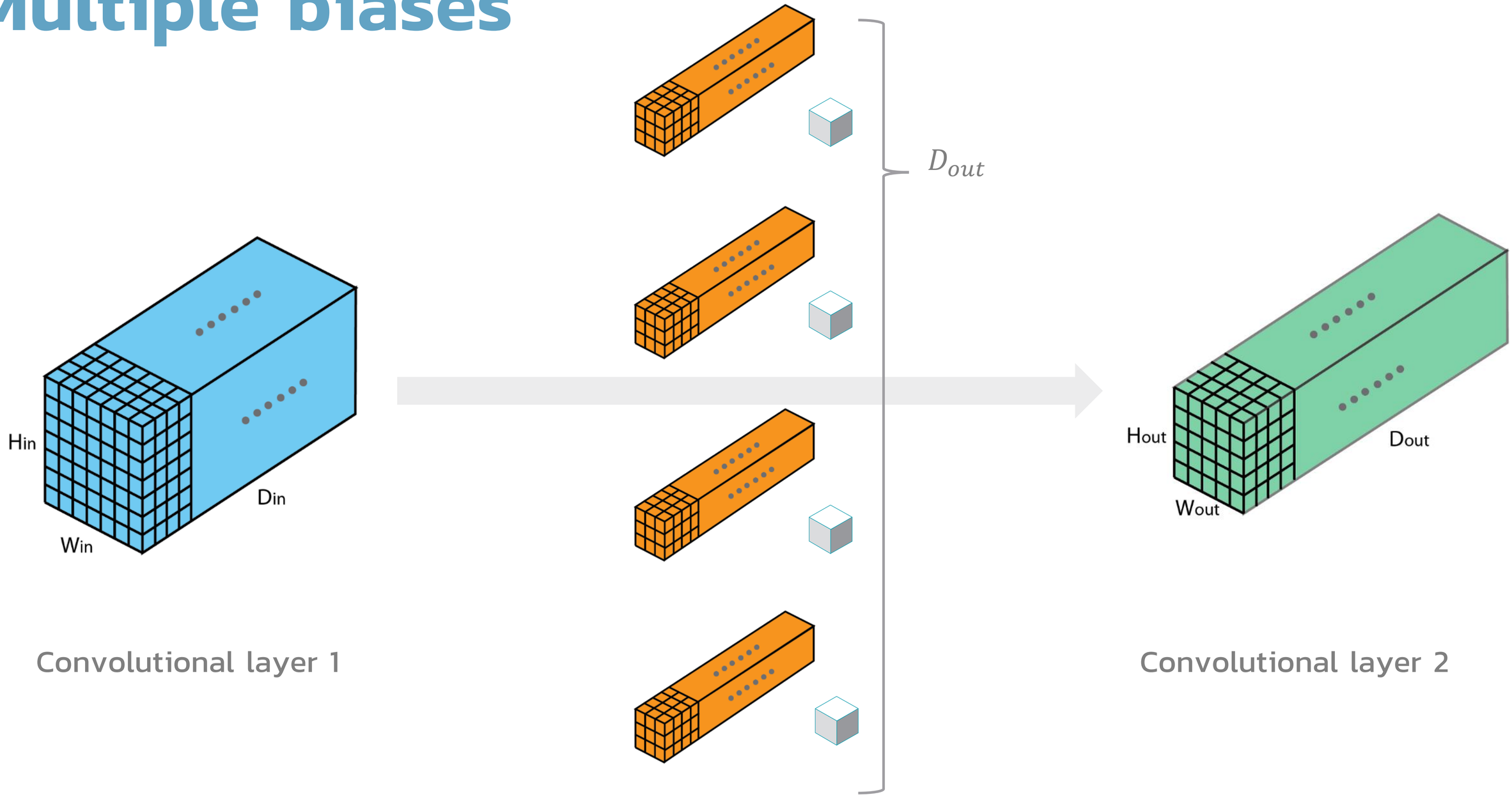
Bias



Multiple filters

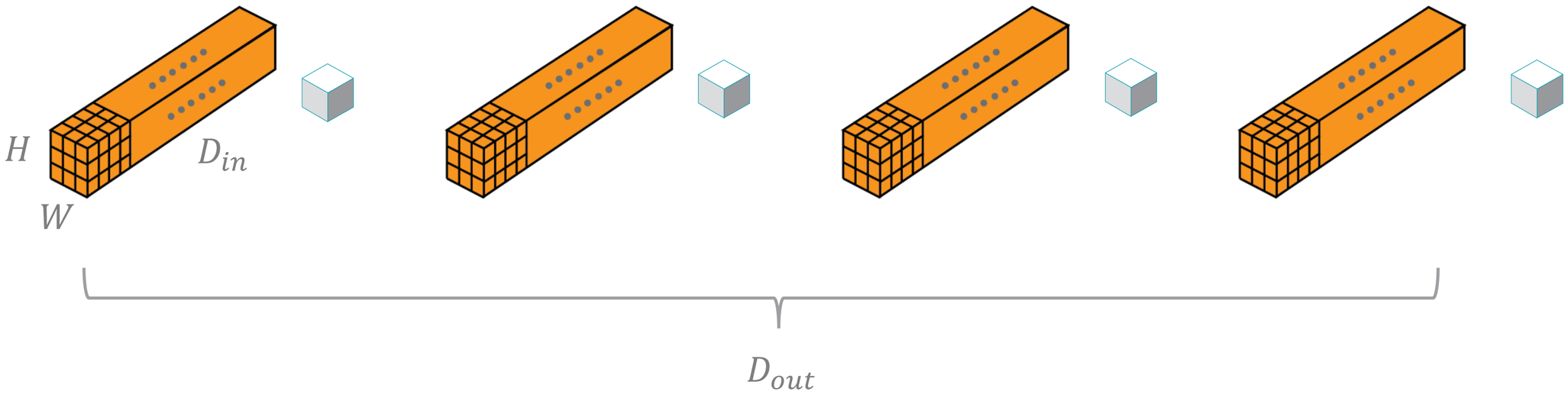


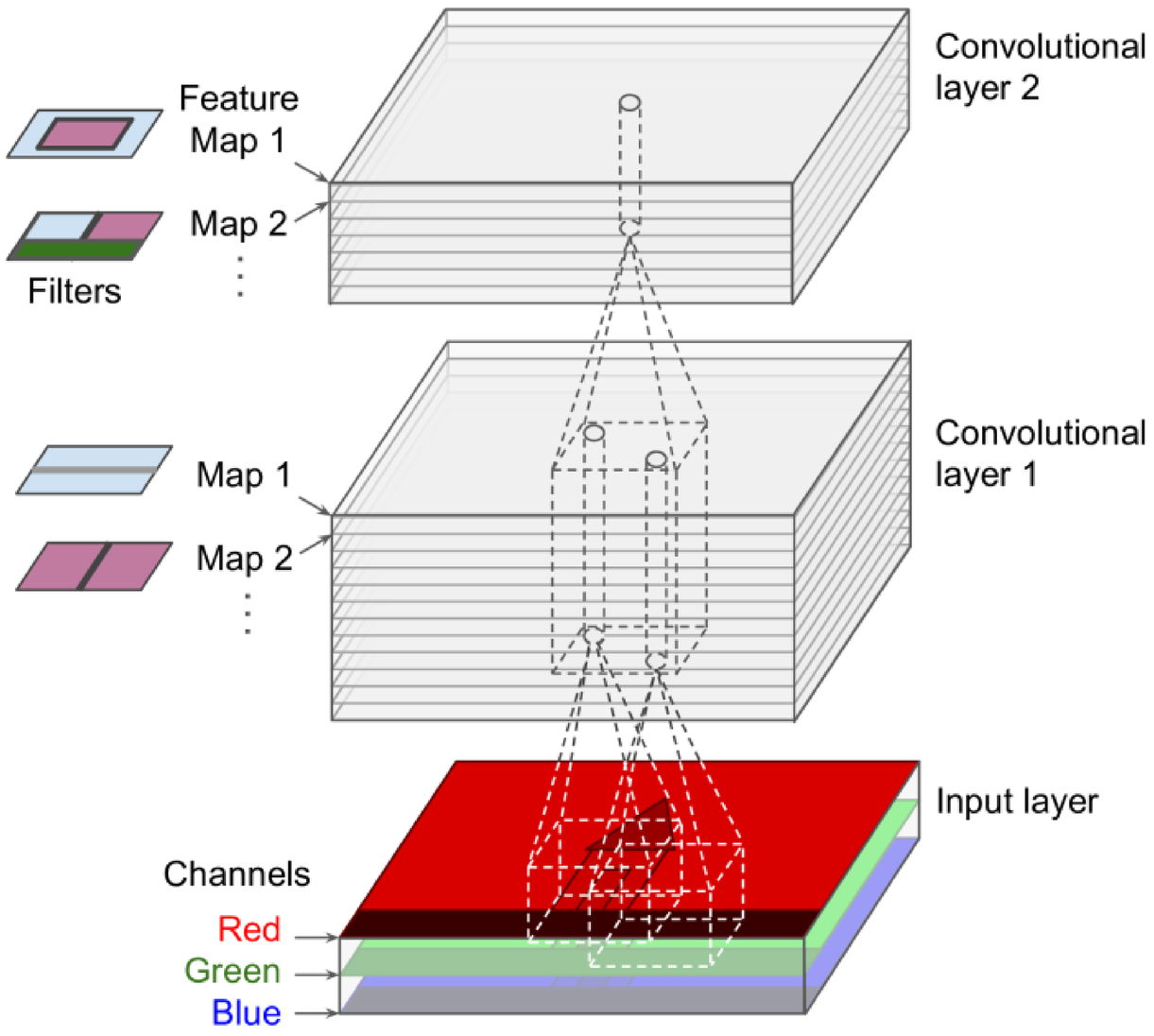
Multiple biases



Dimensions

- Filter: 4D array
  - Width x Height x D_{in} x D_{out}
- Bias: 1D array
  - D_{out}

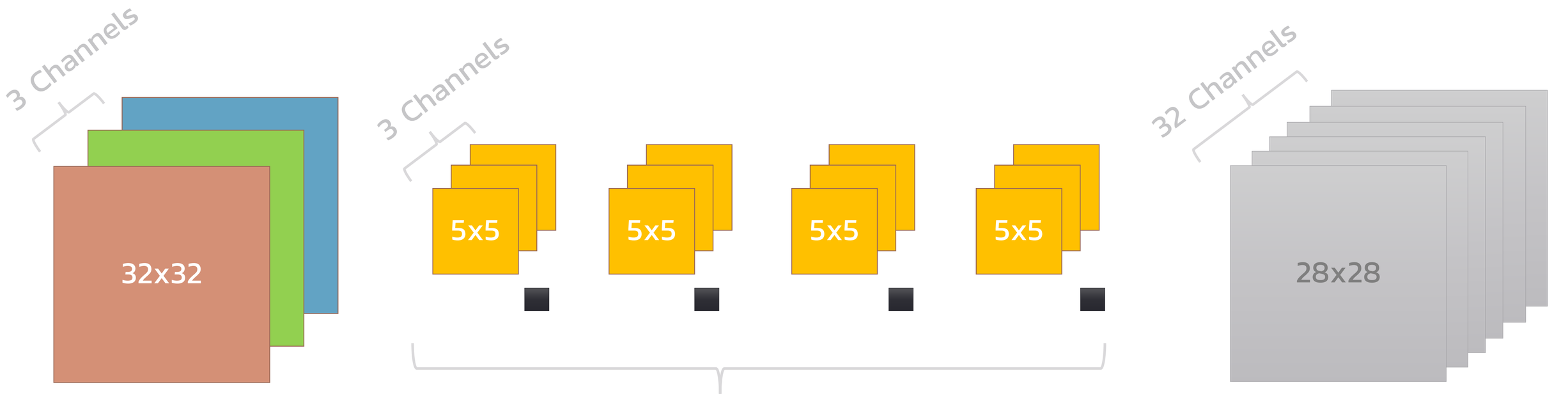



Feature
Map 1
Map 2
Filters
Convolutional
layer 2
Map 1
Map 2
Convolutional
layer 1
Input layer
Channels
Red
Green
Blue

Exercise

- Image
  - Size: 32x32
  - Number of channels: 3
- Kernel
  - Size: 5x5
  - Number of filters: 32
  - Padding: valid (0, 0)
  - Stride: (1, 1)

1. *How many parameters to train?*
2. *What is the output size?*

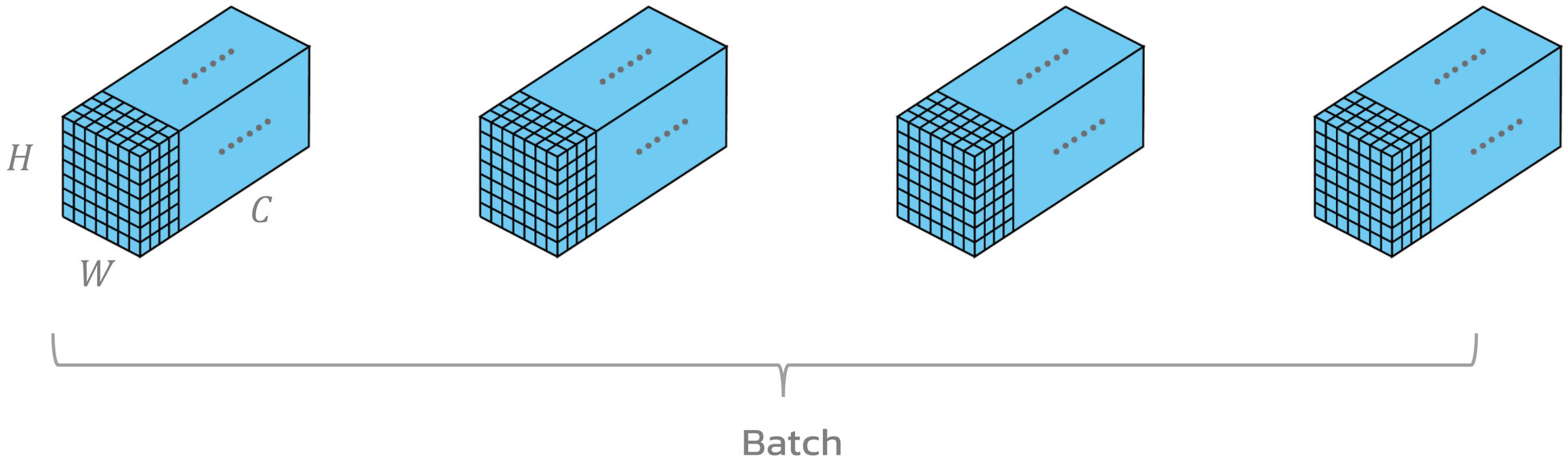


$$Size = Floor\left[\frac{(32 - 2 \times 0 - 5)}{1}\right] + 1 = 28$$

- Number of parameters = $[3 \times (5 \times 5) + 1] \times 32 = 2432$
- Output size = $28 \times 28 \times 32$

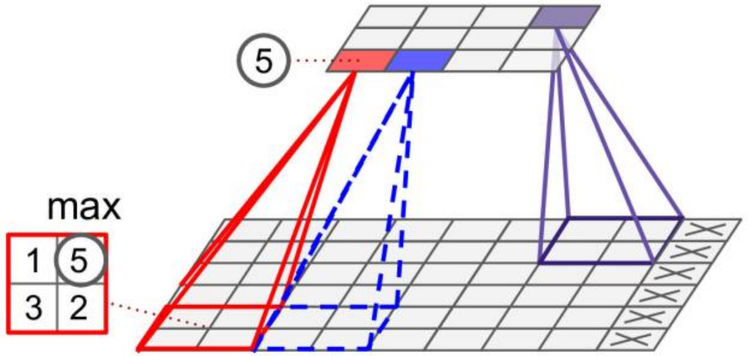
Multiple images

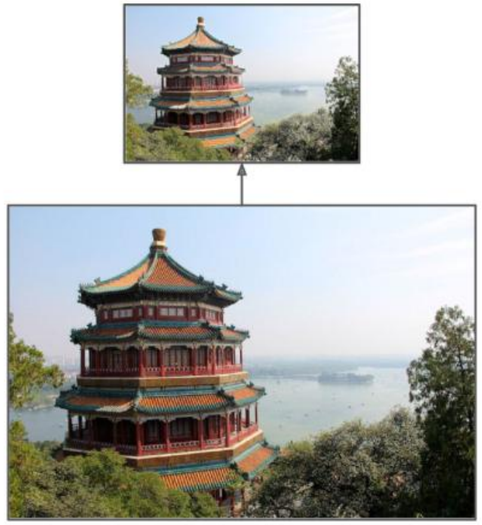
- Neurons in each layer becomes 4D arrays.
  - Batch x Width x Height x Channel



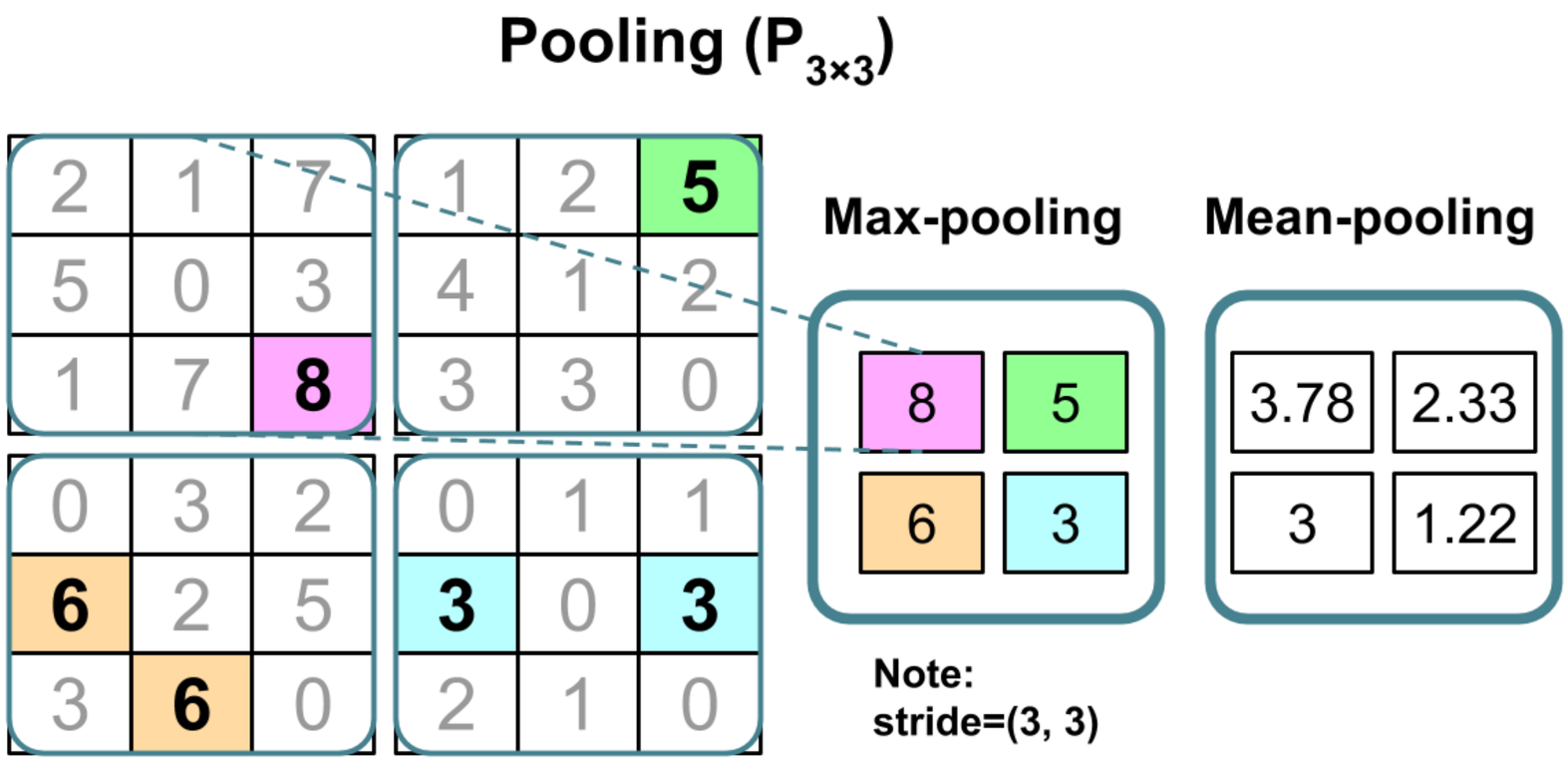
Pooling layer

- Subsample neurons in the previous layer
  - Using aggregate function such as *max* or *mean*.
- Has no weight.
- Characterize by size, stride, padding type.
- Most common is max pooling layer.





Pooling layer



Advantages of pooling layers

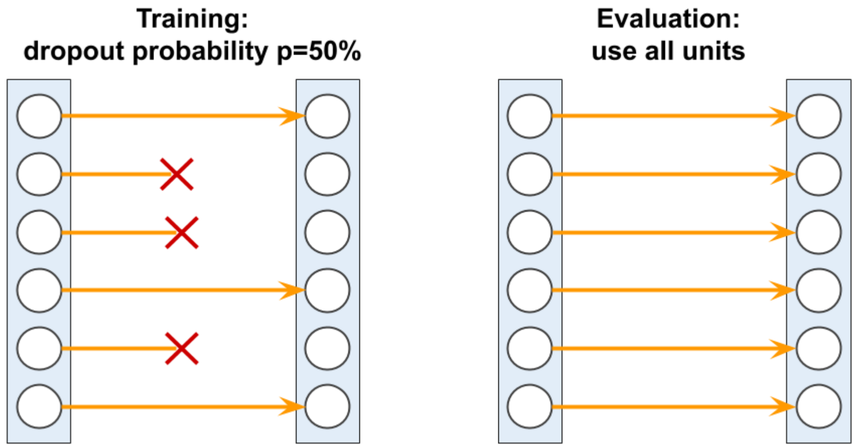
- Decrease computational expense
  - Reduce feature sizes.
- Local invariance
  - Small change in local neighborhood does not change the result of the max pool.

$$\left.\begin{array}{l} \boldsymbol{X}_1 = \begin{bmatrix} 10 & 255 & 125 & 0 & 170 & 100 \\ 70 & 255 & 105 & 25 & 25 & 70 \\ 255 & 0 & 150 & 0 & 10 & 10 \\ 0 & 255 & 10 & 10 & 150 & 20 \\ 70 & 15 & 200 & 100 & 95 & 0 \\ 35 & 25 & 100 & 20 & 0 & 60 \end{bmatrix} \\ \\ \boldsymbol{X}_2 = \begin{bmatrix} 100 & 100 & 100 & 50 & 100 & 50 \\ 95 & 255 & 100 & 125 & 125 & 170 \\ 80 & 40 & 10 & 10 & 125 & 150 \\ 255 & 30 & 150 & 20 & 120 & 125 \\ 30 & 30 & 150 & 100 & 70 & 70 \\ 70 & 30 & 100 & 200 & 70 & 95 \end{bmatrix} \end{array}\right\} \xrightarrow{\text{max pooling } P_{2\times 2}} \begin{bmatrix} 255 & 125 & 170 \\ 255 & 150 & 150 \\ 70 & 200 & 95 \end{bmatrix}$$

- *Assuming pool size = (2, 2) and no overlap*
- *Small change in X does not affect the output*

Dropout layer

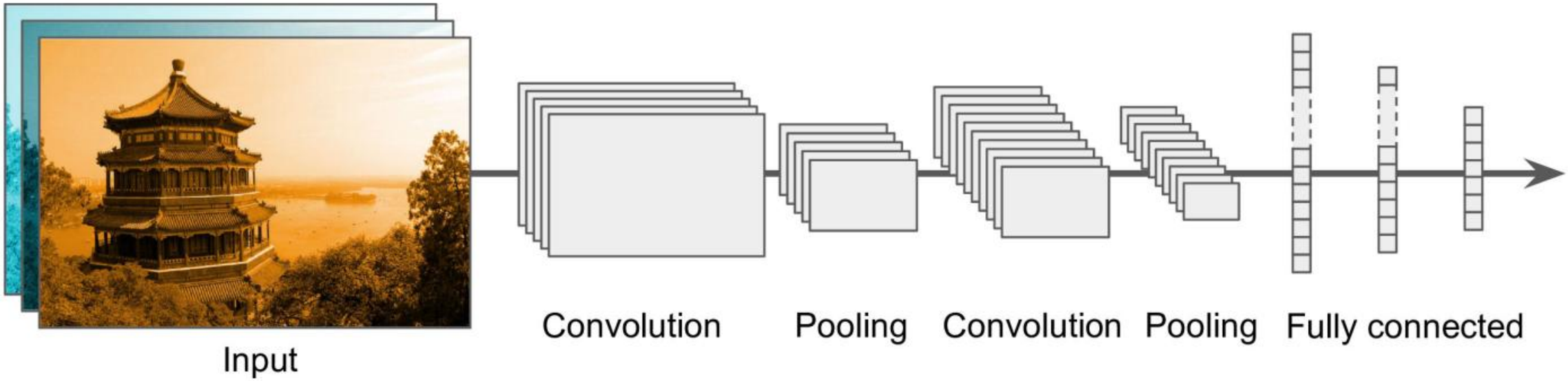
- Fraction of hidden units are randomly dropped with playability p_{drop}
  - Weight of the neurons are rescaled to account for missing neurons.
- Network is forced to learn redundant representation of the data.
  - Network cannot rely on activation of any set of hidden units.
- During evaluate, no dropout is used.



Loss function

Loss function	Usage	Examples	
		Using probabilities *from_logits=False*	Using logits *from_logits=True*
`BinaryCrossentropy`	Binary classification	y_true: 1 y_pred: 0.69	y_true: 1 y_pred: 0.8
`CategoricalCrossentropy`	Multiclass classification	y_true: 0 0 1 y_pred: 0.30 0.15 0.55	y_true: 0 0 1 y_pred: 1.5 0.8 2.1
`Sparse CategoricalCrossentropy`	Multiclass classification	y_true: 2 y_pred: 0.30 0.15 0.55	y_true: 2 y_pred: 1.5 0.8 2.1

Basic structure



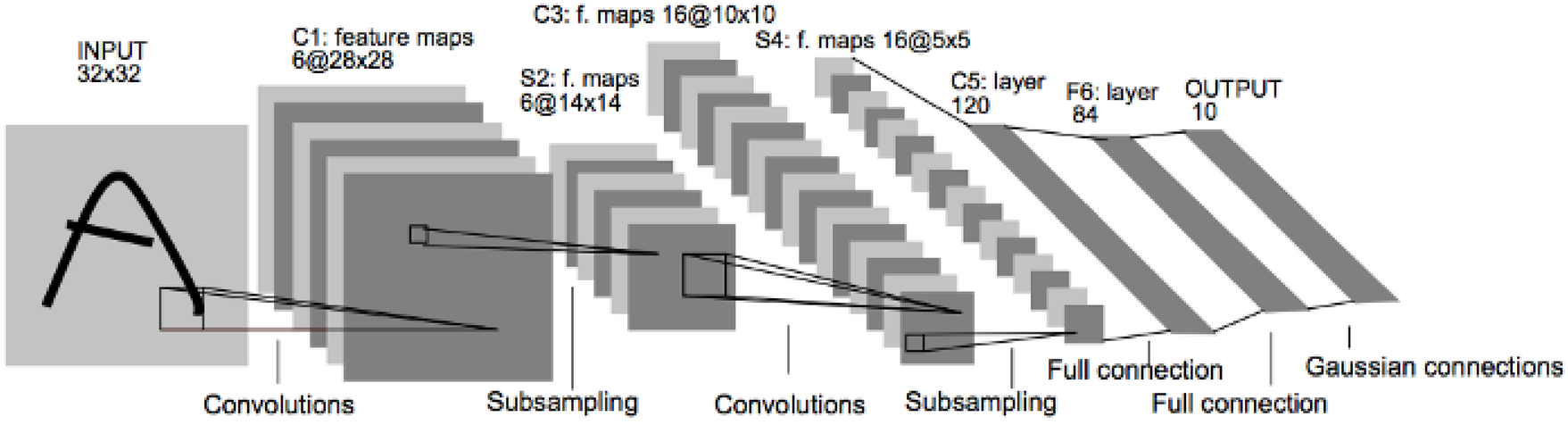
Visualization

- https://poloclub.github.io/cnn-explainer

CNN Architectures

LeNet-5

- Most widely known CNN architecture.
- Create by Yann LeCun in 1998.
- Used for hand written digit recnognition.
  - MNIST.



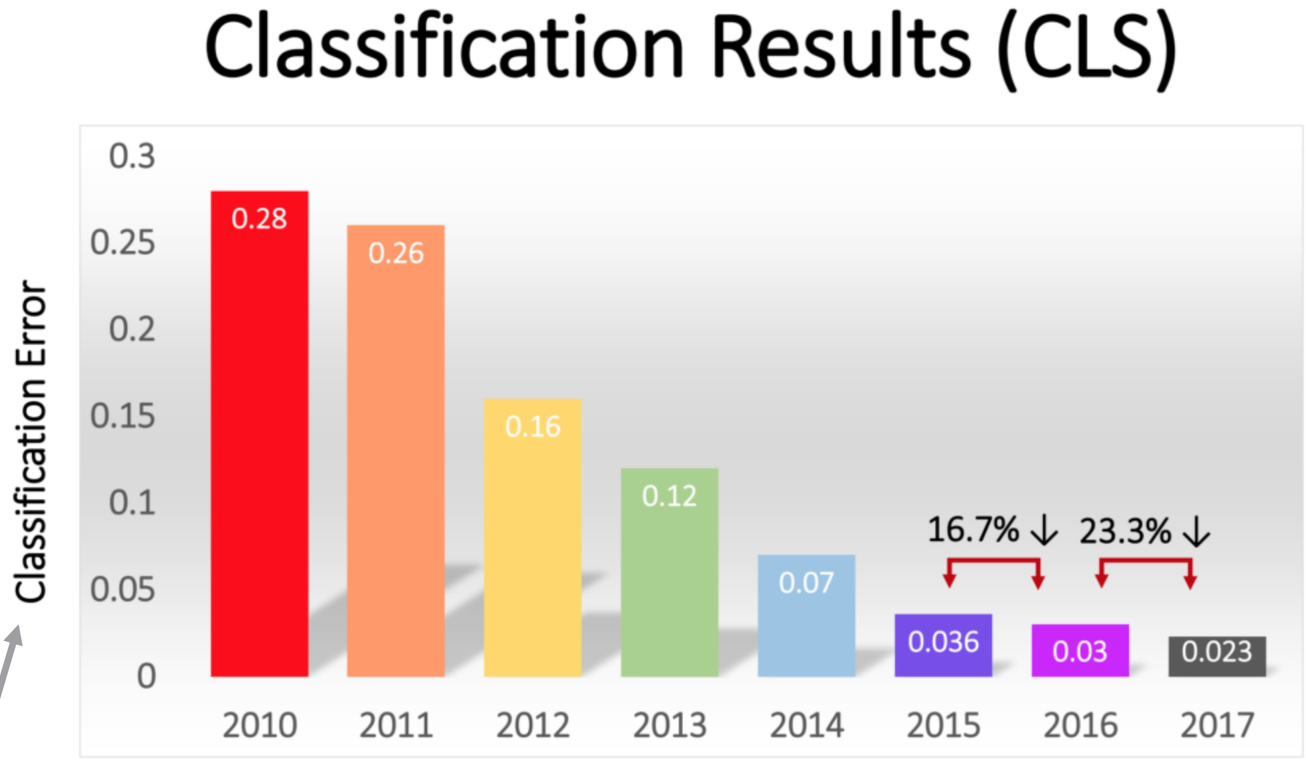
Layer	Type	Maps	Size	Kernel size	Stride	Activation
Out	Fully Connected	–	10	–	–	RBF
F6	Fully Connected	–	84	–	–	tanh
C5	Convolution	120	1×1	5×5	1	tanh
S4	Avg Pooling	16	5×5	2×2	2	tanh
C3	Convolution	16	10×10	5×5	1	tanh
S2	Avg Pooling	6	14×14	2×2	2	tanh
C1	Convolution	6	28×28	5×5	1	tanh
In	Input	1	32×32	–	–	–

ILSVRC competition

- ImageNet Large Scale Visual Recognition Challenge
  - 2010-2017 (8 years)
- Task
  - Evaluates algorithms for object detection and image classification at large scale.
- Data
  - 15 million images
  - 22000 categories

https://www.image-net.org/challenges/LSVRC/

ILSVRC result

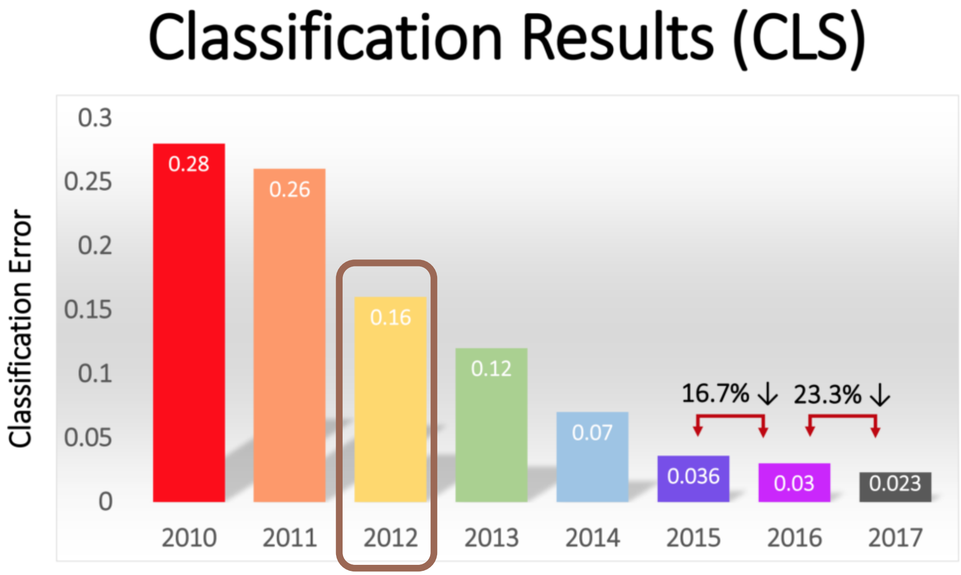


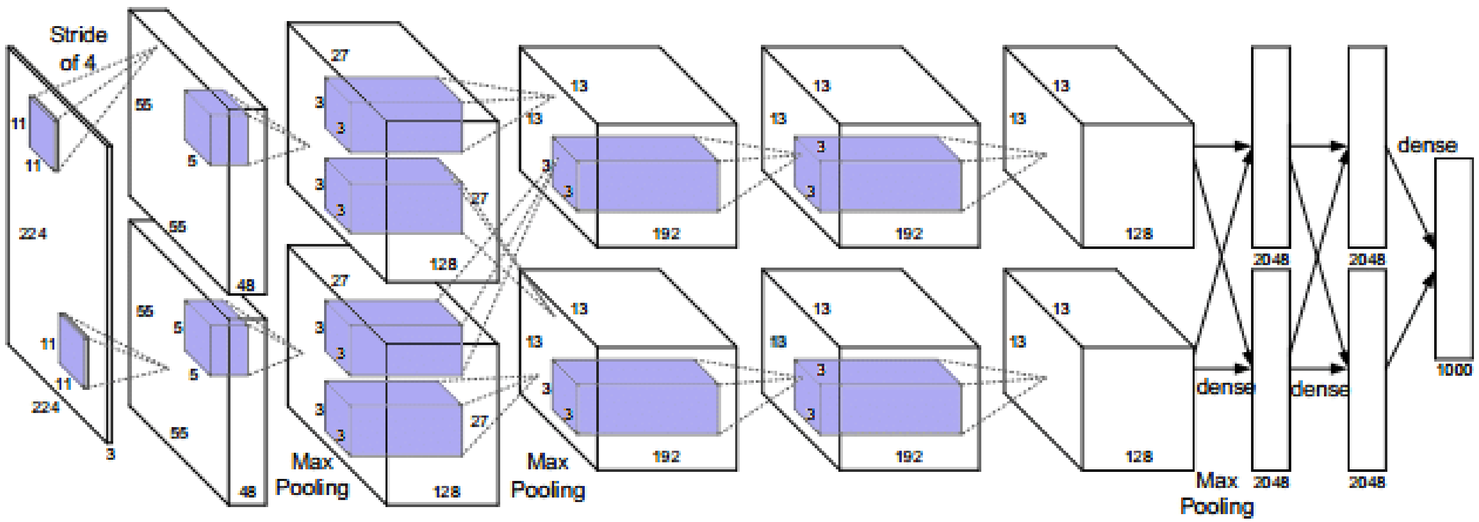
Top-5 Error

Exceed human ability

AlexNet (2012)

- Won 2012 ILSVRC challenge.
- Developed by Alex Krizhevsky, *et al.*
- Much deeper than LeNet-5.

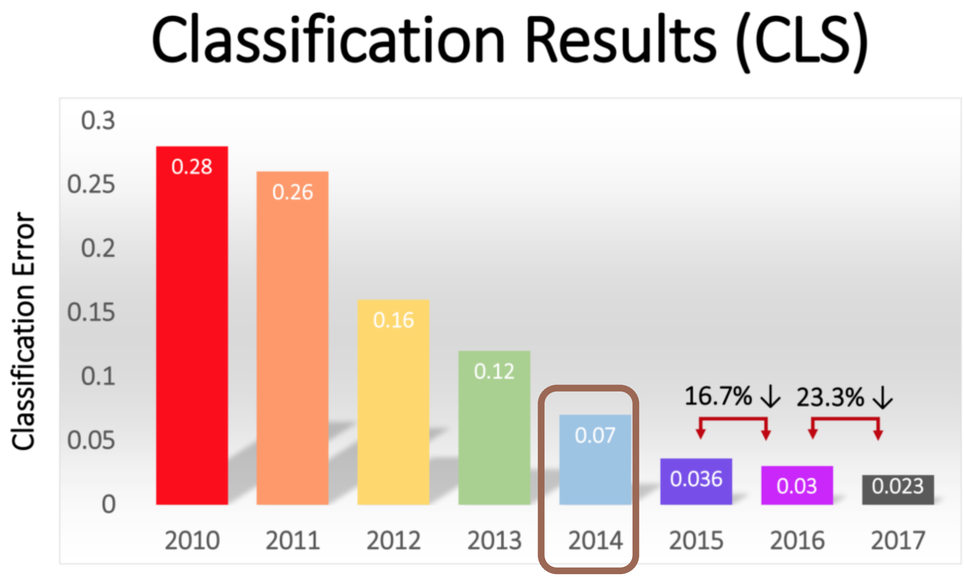


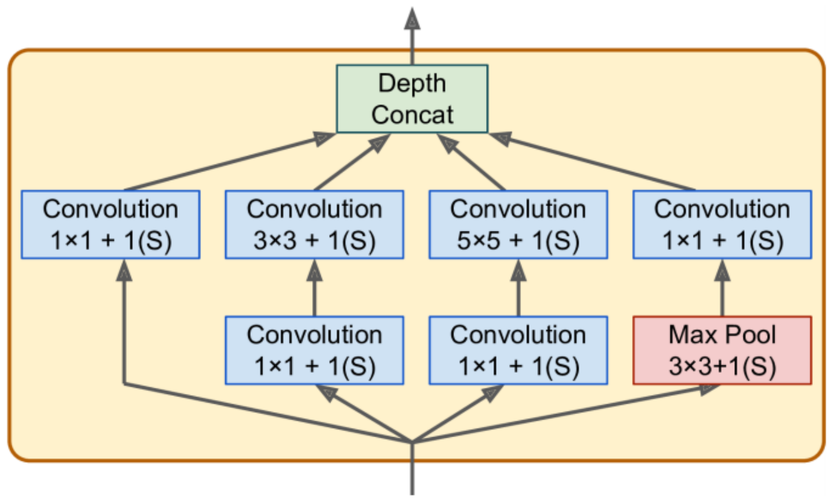


Layer	Type	Maps	Size	Kernel size	Stride	Padding	Activation
Out	Fully Connected	–	1,000	–	–	–	Softmax
F9	Fully Connected	–	4,096	–	–	–	ReLU
F8	Fully Connected	–	4,096	–	–	–	ReLU
C7	Convolution	256	13×13	3×3	1	SAME	ReLU
C6	Convolution	384	13×13	3×3	1	SAME	ReLU
C5	Convolution	384	13×13	3×3	1	SAME	ReLU
S4	Max Pooling	256	13×13	3×3	2	VALID	–
C3	Convolution	256	27×27	5×5	1	SAME	ReLU
S2	Max Pooling	96	27×27	3×3	2	VALID	–
C1	Convolution	96	55×55	11×11	4	VALID	ReLU
In	Input	3 (RGB)	227×227	–	–	–	–

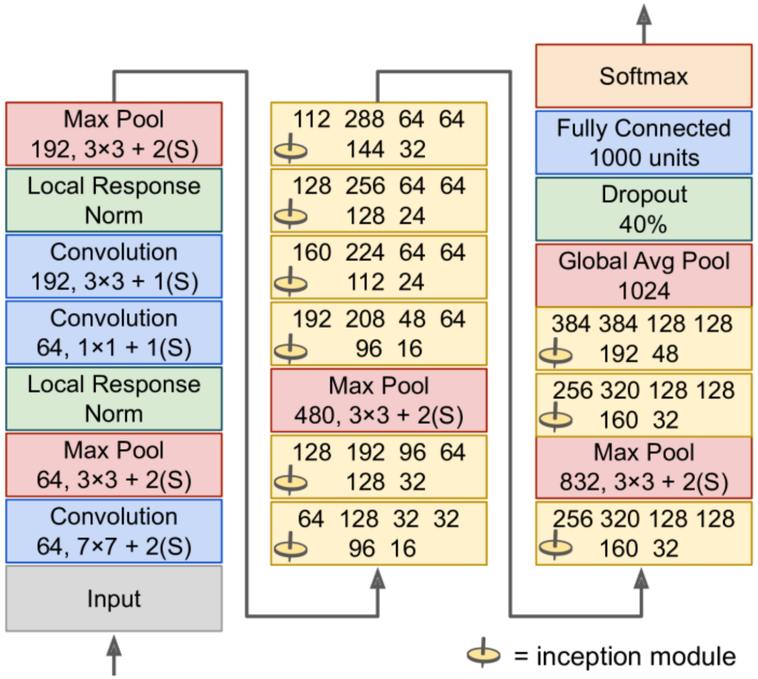
GoogLeNet

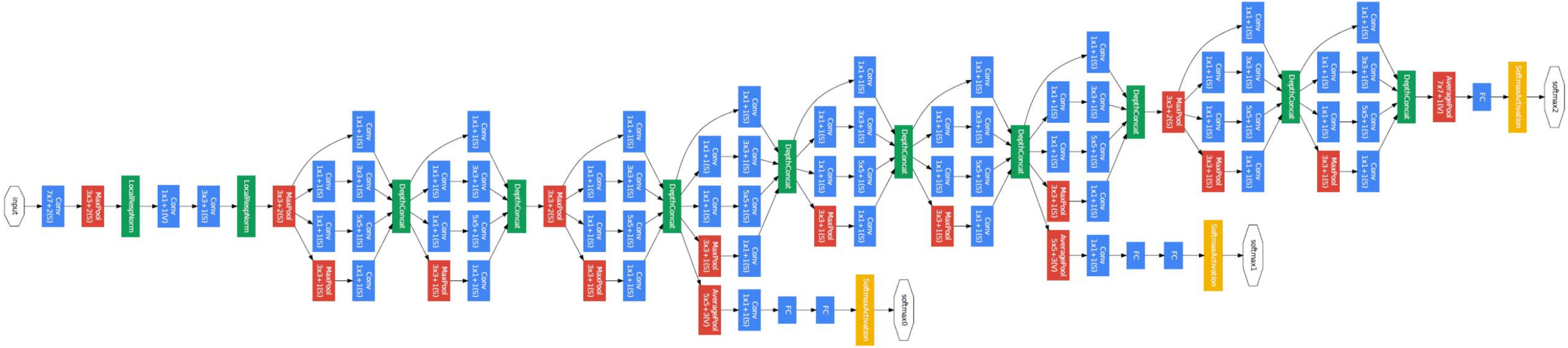
- Won ILSVRC 2014 challenge.
- Developed by Christian Szegedy et al. from Google Research.
- Much deeper network
- Use subnetwork called inception modules.
  - Much more efficient.





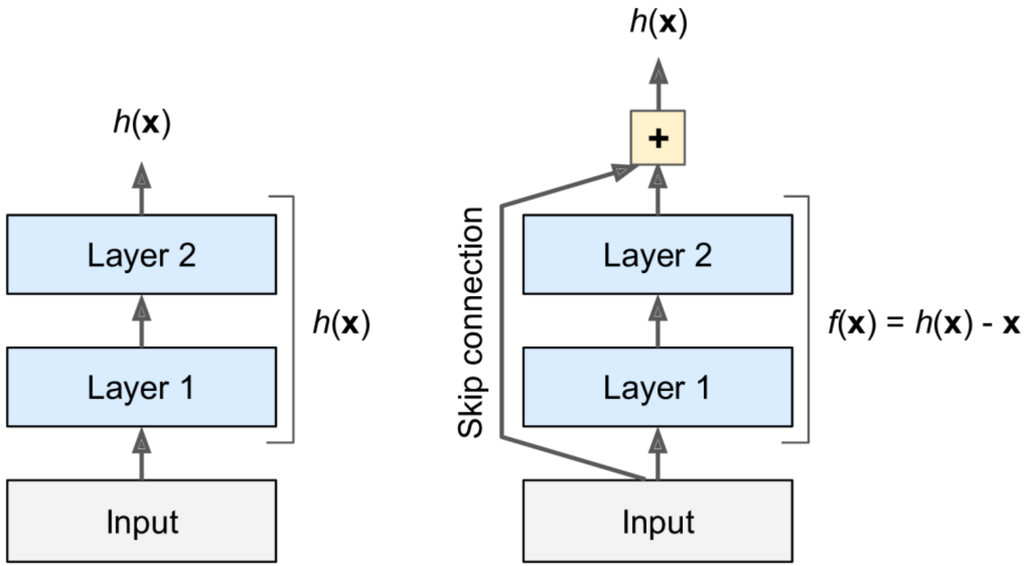
Inception Modules


Softmax
Fully Connected
1000 units
Dropout
40%
Global Avg Pool
1024
384 384 128 128
192 48
256 320 128 128
160 32
Max Pool
832, 3×3 + 2(S)
256 320 128 128
160 32
112 288 64 64
144 32
128 256 64 64
128 24
160 224 64 64
112 24
192 208 48 64
96 16
Max Pool
480, 3×3 + 2(S)
128 192 96 64
128 32
64 128 32 32
96 16
Max Pool
192, 3×3 + 2(S)
Local Response
Norm
Convolution
192, 3×3 + 1(S)
Convolution
64, 1×1 + 1(S)
Local Response
Norm
Max Pool
64, 3×3 + 2(S)
Convolution
64, 7×7 + 2(S)
Input
= inception module

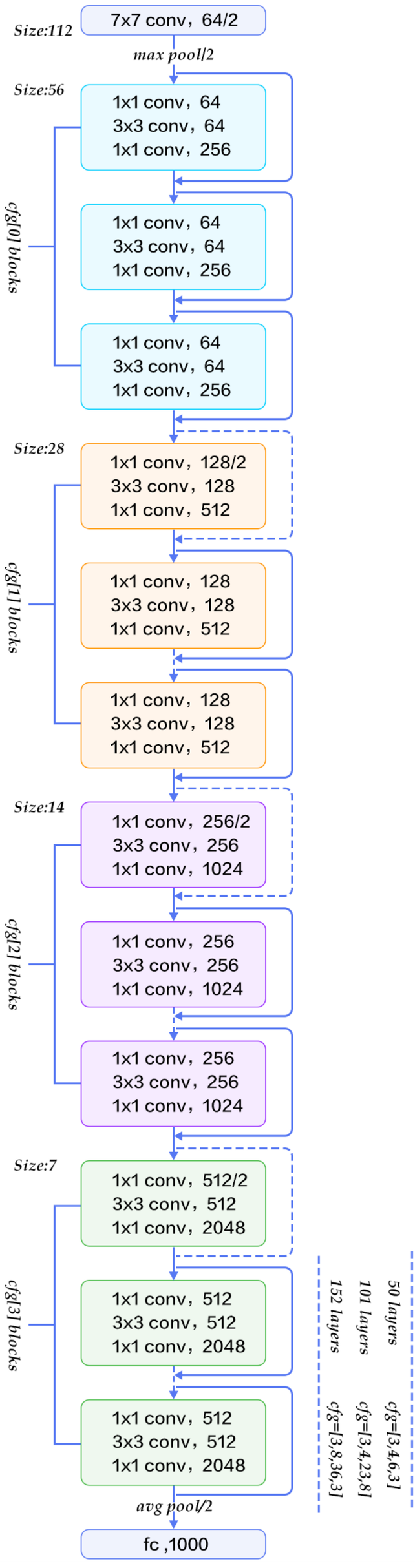

input
Conv 7x7+2(S)
MaxPool 3x3+2(S)
LocalRespNorm
Conv 1x1+1(V)
Conv 3x3+1(S)
LocalRespNorm
MaxPool 3x3+2(S)
Conv 1x1+1(S)
Conv 3x3+1(S)
Conv 5x5+1(S)
MaxPool 3x3+1(S)
Conv 1x1+1(S)
DepthConcat
Conv 1x1+1(S)
Conv 3x3+1(S)
Conv 5x5+1(S)
MaxPool 3x3+1(S)
Conv 1x1+1(S)
DepthConcat
MaxPool 3x3+2(S)
Conv 1x1+1(S)
Conv 3x3+1(S)
Conv 5x5+1(S)
MaxPool 3x3+1(S)
Conv 1x1+1(S)
DepthConcat
Conv 1x1+1(S)
Conv 3x3+1(S)
Conv 5x5+1(S)
MaxPool 3x3+1(S)
Conv 1x1+1(S)
AveragePool 5x5+3(V)
Conv 1x1+1(S)
DepthConcat
Conv 1x1+1(S)
Conv 3x3+1(S)
Conv 5x5+1(S)
MaxPool 3x3+1(S)
Conv 1x1+1(S)
DepthConcat
Conv 1x1+1(S)
Conv 3x3+1(S)
Conv 5x5+1(S)
MaxPool 3x3+1(S)
Conv 1x1+1(S)
DepthConcat
Conv 1x1+1(S)
Conv 3x3+1(S)
Conv 5x5+1(S)
MaxPool 3x3+1(S)
Conv 1x1+1(S)
AveragePool 5x5+3(V)
Conv 1x1+1(S)
DepthConcat
Conv 1x1+1(S)
Conv 3x3+1(S)
Conv 5x5+1(S)
MaxPool 3x3+1(S)
Conv 1x1+1(S)
DepthConcat
MaxPool 3x3+2(S)
Conv 1x1+1(S)
Conv 3x3+1(S)
Conv 5x5+1(S)
MaxPool 3x3+1(S)
Conv 1x1+1(S)
DepthConcat
Conv 1x1+1(S)
Conv 3x3+1(S)
Conv 5x5+1(S)
MaxPool 3x3+1(S)
Conv 1x1+1(S)
DepthConcat
AveragePool 7x7+1(V)
FC
SoftmaxActivation
softmax2
FC
FC
SoftmaxActivation
softmax1
FC
FC
SoftmaxActivation
softmax0

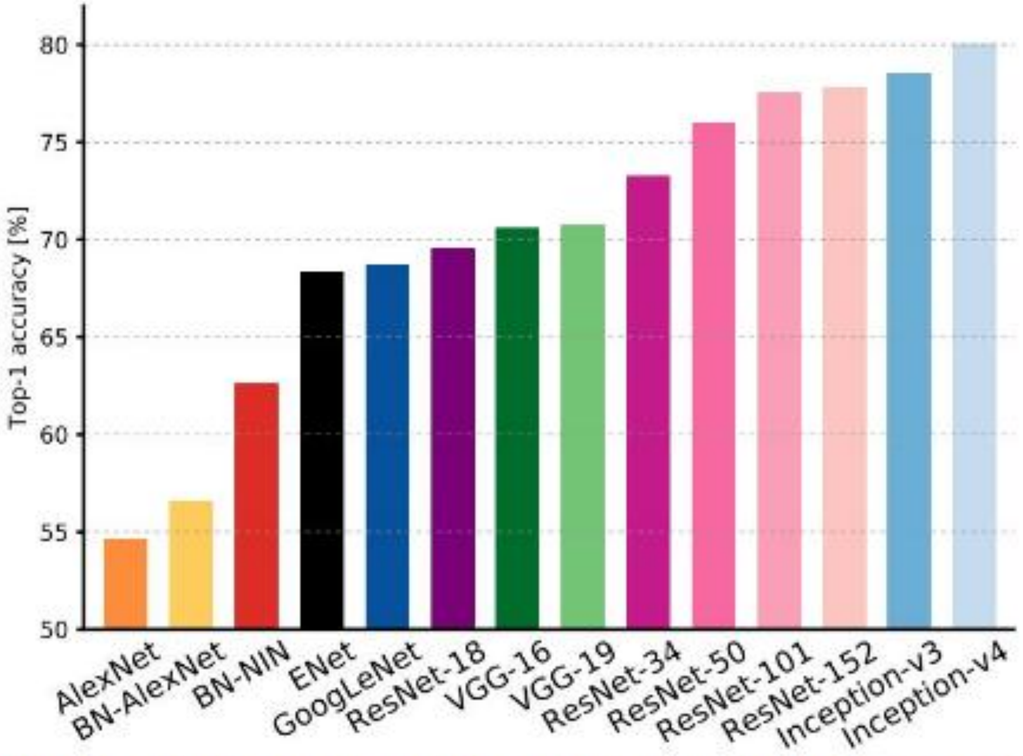
ResNet

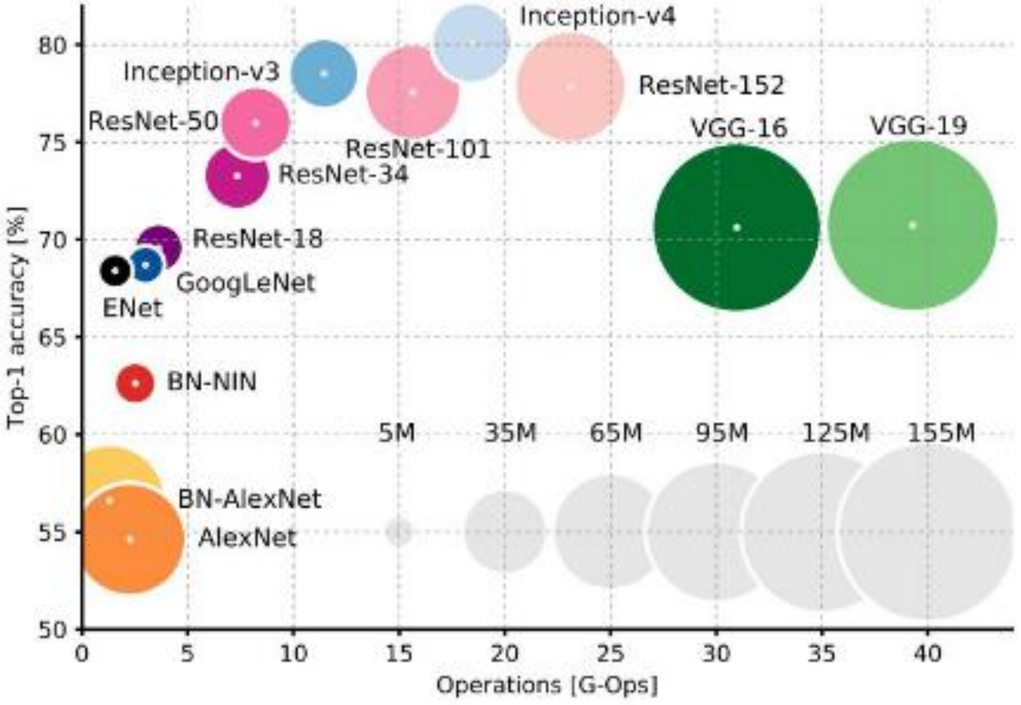
- Won ILSVRC 2015 Challenge.
- Developed by Kaiming He et al.
- 152 layers
- Use skip connection
  - Residual learning



Residual learning


7x7 conv，64/2
max pool/2
Size:112
Size:56
1x1 conv，64
3x3 conv，64
1x1 conv，256
cfg[0] blocks
Size:28
1x1 conv，128/2
3x3 conv，128
1x1 conv，512
cfg[1] blocks
Size:14
1x1 conv，256/2
3x3 conv，256
1x1 conv，1024
cfg[2] blocks
Size:7
1x1 conv，512/2
3x3 conv，512
1x1 conv，2048
cfg[3] blocks
avg pool/2
fc ,1000
50 layers cfg=[3,4,6,3]
101 layers cfg=[3,4,23,8]
152 layers cfg=[3,8,36,3]





An Analysis of Deep Neural Network Models for Practical Applications, 2017.

What Happens Now?

IMAGENET + kaggle

ImageNet **Object Localization** Challenge

ImageNet **Object Detection** Challenge

ImageNet **Object Detection from Video** Challenge

Current benchmark

- https://paperswithcode.com/sota/image-classification-on-imagenet